

Relatorio Tecnico

Sistema Escola/Curso - Programacao Orientada a Objetos

Projeto Escola

Este relatorio apresenta as principais decisoes tecnicas, funcionalidades implementadas, dificuldades encontradas e metricas de qualidade do sistema escolar desenvolvido em Java. O projeto foi organizado para demonstrar conceitos de Programacao Orientada a Objetos em um dominio academico, com cadastro de entidades, matriculas, turmas e avaliacao por professor responsavel.

1. Decisoes de Projeto

Justificativas para escolhas arquiteturais e de design

- A aplicacao foi desenvolvida em Java com interface de console, mantendo o foco nos conceitos de POO e nas regras de negocio do dominio escolar.
- A estrutura foi separada em pacotes: model, service, repository, exception, util e main. Essa divisao facilita a leitura e separa dominio, regras, armazenamento em memoria, excecoes e interacao com o usuario.
- O armazenamento foi mantido em memoria por meio de repositorios, conforme a proposta do trabalho, sem criacao de banco de dados.
- A responsabilidade de avaliacao foi associada ao Professor. A classe Professor implementa a interface Avaliavel, deixando claro que o professor e quem lanca e lista notas.
- A classe Turma possui um professor responsavel, e a classe Matricula liga Aluno e Turma. Assim, antes de lancar nota, o sistema consegue validar se o professor informado e realmente responsavel pela turma da matricula.
- Os IDs sao gerados automaticamente, reduzindo erros de digitacao e tornando o cadastro mais proximo de um sistema real.

Relatorio Tecnico

Funcionalidades e regras de negocio

2. Funcionalidades Implementadas

Lista de features desenvolvidas e testadas

- Gerenciamento de cursos: cadastrar, listar, atualizar, remover, ativar, desativar e acessar disciplinas vinculadas.
- Gerenciamento de disciplinas: cadastrar em um curso, listar, atualizar, remover, ativar, desativar, gerenciar turmas e professores aptos.
- Gerenciamento de turmas: cadastrar turmas em disciplinas, listar, atualizar, remover, ativar, desativar, definir professor responsavel e listar alunos.
- Gerenciamento de alunos e professores: cadastro, listagem, atualizacao e remocao, com dados pessoais e informacoes especificas.
- Gerenciamento de matriculas: realizar matricula, confirmar, cancelar e consultar vinculo entre aluno e turma.
- Lancamento de notas dentro do menu Gerenciar Professores, reforcando que a avaliacao e comportamento do professor.
- Listagem de notas de uma turma pelo professor responsavel.
- Listagem polimorfica de pessoas cadastradas no sistema, tratando alunos e professores como Pessoa.

Validacoes principais

- A nota precisa estar entre 0 e 10; valores fora desse intervalo geram `NotaInvalidaException`.
- A matricula precisa estar confirmada para receber nota.
- Matriculas canceladas nao podem receber nota.
- A turma precisa possuir professor definido.
- O professor informado precisa ser o professor responsavel pela turma.

Relatorio Tecnico

Dificuldades e qualidade

3. Dificuldades Encontradas

Desafios tecnicos e como foram superados

- A principal decisao de dominio foi mover as notas para o contexto de Professor, evitando um menu independente de Gerenciar notas e deixando o fluxo mais coerente com a regra escolar.
- Foi necessario validar a relacao entre professor, turma e matricula para impedir que um professor lance nota em turma que nao e sua.
- A remocao do NotaService exigiu verificar dependencias para nao quebrar o sistema, ja que a responsabilidade passou a ser da classe Professor por meio da interface Avaliavel.
- Foram corrigidos textos e adotada a compilacao com encoding adequado para evitar problemas no terminal ou na IDE.
- A apresentacao do UML foi ajustada para destacar Professor implementando Avaliavel, alem das relacoes entre Turma, Matricula, Aluno e Professor.

4. Metricas de Qualidade

Compilacao, testes manuais e organizacao do codigo

- Compilacao validada com javac, sem erros de compilacao.
- Projeto composto por 24 arquivos Java e aproximadamente 2.198 linhas de codigo.
- Fluxo principal testado manualmente: cadastro de entidades, criacao de turma, definicao de professor, matricula, confirmacao da matricula, lancamento de nota e listagem de notas.
- Testes manuais tambem cobriram regras negativas, como nota invalida e tentativa de lancar nota com professor que nao e responsavel pela turma.
- Uso de excecoes personalizadas para situacoes de dominio, como NotaInvalidaException e MatriculaInvalidaException.
- Organizacao por camadas e pacotes mantem o projeto mais legivel para manutencao e apresentacao.

Conclusao

O sistema está funcional para a apresentação final e demonstra herança, interfaces, polimorfismo, encapsulamento, associações e tratamento de exceções.